WEB TECHNOLOGY

UNIT IV – REPRESENTING WEB DATA

ASSESSMENT I: DATA INTERPRETATION – ANSWER KEY

Duration	60 Minutes	**Maximum Marks**	30
Mode	Individual Assessment	**Unit**	IV

Question 1: Interpret the XML Structure – 5 Marks

(1 mark each)

a. Identify the root element.

Answer: The root element is **`<courses>`**.

b. Identify the repeating record element.

Answer: The repeating record element is **`<course>`**. Each course is represented as a separate `<course>` element under the root.

c. Identify the attribute used to uniquely identify each course.

Answer: The attribute **`id`** (e.g., `id="C101"`, `id="C102"`, etc.) uniquely identifies each course.

d. Identify the elements that represent numeric information.

Answer: The elements that represent numeric information are:

- **`<students>`** – stores the number of students enrolled
- **`<credits>`** – stores the credit value of the course

e. State whether the XML document is structurally well-formed and justify your answer.

Answer: Yes, the XML document is structurally well-formed.

Justification:

- It has a single root element (`<courses>`).
- All tags are properly nested and correctly closed.
- Attribute values are enclosed in double quotes.
- There are no overlapping or mismatched tags.
- The XML declaration is present and correctly written.

Question 2: Apply XPath for Data Selection – 10 Marks

(1 mark each)

a. All course records.

XPath: `/courses/course`

b. Names of all courses.

XPath: `/courses/course/name`

c. Courses having more than 50 students.

XPath: `/courses/course[students > 50]`

d. Courses carrying 4 credits.

XPath: `/courses/course[credits = 4]`

e. Courses whose type is Theory.

XPath: `/courses/course[type = 'Theory']`

f. Names of Theory courses having more than 50 students.

XPath: `/courses/course[type = 'Theory' and students > 50]/name`

g. Faculty members handling courses with at least 4 credits.

XPath: `/courses/course[credits >= 4]/faculty`

h. The course whose id is C104.

XPath: `/courses/course[@id = 'C104']`

i. The first course available in the XML document.

XPath: `/courses/course[1]`

j. The last course available in the XML document.

XPath: `/courses/course[last()]`

Question 3: Apply XSLT for Data Presentation – 10 Marks

(Covers filtering, sorting, heading, XPath condition, and valid HTML)

Complete XSLT Stylesheet:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:output method="html" indent="yes"/>

  <xsl:template match="/">
    <html>
      <head>
        <title>High Enrollment Courses</title>
        <style>
          table { border-collapse: collapse; width: 90%; margin: 20px auto; }
          th, td { border: 1px solid #333; padding: 8px; text-align: left; }
          th { background-color: #2b6cb0; color: white; }
          h2 { text-align: center; color: #1a365d; }
        </style>
      </head>
      <body>
        <h2>High Enrollment Courses</h2>
        <table>
          <tr>
            <th>Course Code</th>
            <th>Course Name</th>
            <th>Faculty</th>
            <th>Students</th>
            <th>Credits</th>
            <th>Type</th>
          </tr>
          <!-- XPath condition: students &gt; 40 -->
          <!-- Sorted in descending order of student enrollment -->
          <xsl:for-each select="courses/course[students &gt; 40]">
            <xsl:sort select="students" data-type="number" order="descending"/>
            <tr>
              <td><xsl:value-of select="code"/></td>
              <td><xsl:value-of select="name"/></td>
              <td><xsl:value-of select="faculty"/></td>
              <td><xsl:value-of select="students"/></td>
              <td><xsl:value-of select="credits"/></td>
              <td><xsl:value-of select="type"/></td>
            </tr>
          </xsl:for-each>
        </table>
      </body>
    </html>
  </xsl:template>

</xsl:stylesheet>
```

Explanation of requirements met:

a. Only courses with more than 40 students are displayed using the XPath predicate `[students > 40]`.

b. Courses are sorted in descending order of enrollment using `<xsl:sort select="students" data-type="number" order="descending"/>`.

c. A suitable heading **“High Enrollment Courses”** is displayed using an <h2> element.

d. An XPath condition is used inside the XSLT: `courses/course[students > 40]`.

e. The stylesheet produces valid HTML output (with proper `<html>`, `<head>`, `<body>`, and a well-formed table).

Expected Output (after transformation):

Course Code	Course Name	Faculty	Students	Credits	Type
AI302	Artificial Intelligence	Dr. Meena	72	4	Theory
ML304	Machine Learning	Dr. Priya	64	4	Theory
WEB301	Web Technology	Dr. Arun	58	4	Theory
DB305	Database Systems	Dr. Kumar	42	3	Theory

Note: WEB303 (36 students) is excluded because it does not satisfy students > 40.

Question 4: Interpret the Extracted Data – 5 Marks

(1 mark each)

a. Identify the course with the highest enrollment.

Answer: Artificial Intelligence (AI302) with **72** students.

b. Identify the course with the lowest enrollment.

Answer: Web Technology Laboratory (WEB303) with **36** students.

c. Determine the number of Theory courses.

Answer: There are **4** Theory courses (WEB301, AI302, ML304, DB305).
Only WEB303 is Practical.

d. Identify all courses having exactly 4 credits.

Answer: The courses with exactly 4 credits are:
• WEB301 – Web Technology
• AI302 – Artificial Intelligence
• ML304 – Machine Learning

e. If an additional teaching assistant is assigned to every course with more than 60 students, identify the courses that require additional support.

Answer: The courses that require additional teaching assistant support (students > 60) are:
• **AI302 – Artificial Intelligence** (72 students)
• **ML304 – Machine Learning** (64 students)

Marks Distribution Summary

Criterion	Marks
Correct interpretation of XML structure (Q1)	5
Correct XPath expressions (Q2)	10

Correct XSLT transformation and sorting (Q3)	10
Correct interpretation of results (Q4)	5
Total	**30**

— End of Answer Key —